



Healthcare Key Management System

What Have We Built? — Project Walkthrough & Viva Preparation Guide

SPRING BOOT | JWT & RBAC | AES-256-GCM

Phase 1 · Foundation

Phase 2 · Key Management

Phase 3 · Auth & Security

Phase 4 · AES-256 Encryption

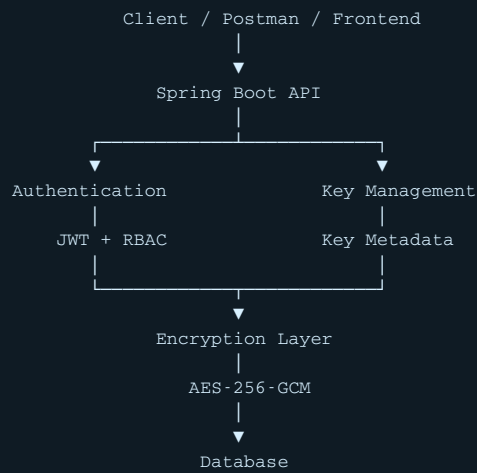
The Problem We're Solving

A healthcare application stores highly sensitive information, including:

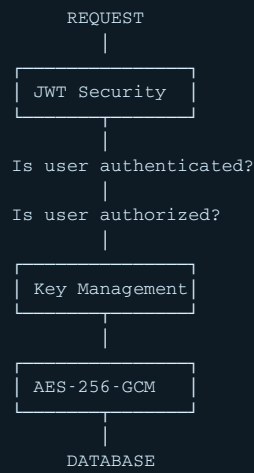
- Patient records
- Medical reports
- Prescriptions
- Personal information
- Healthcare documents

None of this — nor the encryption keys protecting it — should be freely accessible. So this project builds a **secure backend system to manage cryptographic keys and control who can access key-related operations.**

System Architecture



Security Layers (Request Lifecycle)



Phase 1 — Foundation

What we did: built the basic Spring Boot backend and configured Spring Boot, Maven, MySQL, JPA/Hibernate, the database connection, and the project/package structure.

Why: without this infrastructure, there is no backend to build the Key Management System on top of.

Think of it this way — Phase 1 = building the house structure.

Viva Questions

Q: Why did you use Spring Boot?

Spring Boot simplifies the development of production-ready Java backend applications by providing auto-configuration, dependency management, embedded servers, and integration with components such as Spring Security and JPA.

Q: Why JPA?

To simplify database interaction by mapping Java entities to database tables and reducing boilerplate SQL/data-access code.

Phase 2 — Key Management Foundation

This is where the project became an actual Key Management System — we created key-related functionality and metadata.

Example API Response

```
{
  "id": 1,
  "keyId": "AES-KEY-001",
  "algorithm": "AES-256",
  "keyVersion": 1,
  "status": "ACTIVE",
  "createdAt": "...",
  "expiresAt": "...",
  "revokedAt": null
}
```

Key point: we store *metadata* about the key — we never expose the actual secret key material through the API.

Field	Why it matters
keyId	Uniquely references a key without exposing key material
algorithm	Identifies which cryptographic algorithm is in use
keyVersion	Supports key rotation and controlled migration
status	Tracks whether a key is active, expired, or revoked
createdAt / expiresAt / revokedAt	Tracks the full lifecycle of a key

Viva Questions

Q: What is a key ID?

A unique identifier used to reference a cryptographic key without exposing the key material itself.

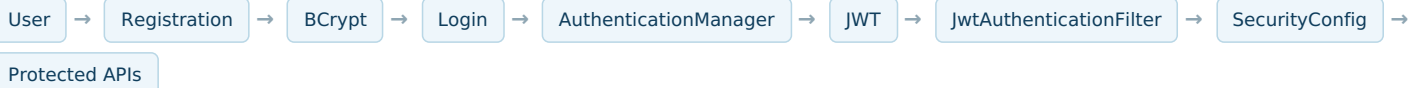
Q: Why do we need key versioning?

Key versioning allows us to rotate cryptographic keys while maintaining a history of previous versions and supporting controlled migration from an old key to a new key.

Q: Why key status?

To track whether a key is active, expired, revoked, or otherwise unavailable for use.

Phase 3 — Authentication & Authorization



Authentication vs Authorization

Authentication	Authorization
Who are you? e.g. username = niraj, password = ***** The system verifies your credentials.	What are you allowed to do? e.g. ADMIN → manage keys USER → limited operations

Authentication identifies the user, while authorization determines what that authenticated user is allowed to access.

Why BCrypt?

Plaintext passwords such as `Password123` are never stored directly. Instead they are hashed:

`Password123` → `BCrypt` → `$2a$10$.....`

BCrypt is specifically designed for password hashing and includes salting/work-factor features that make brute-force attacks more difficult.

⚠ Important: BCrypt is for passwords only. It is **not** what we use to encrypt healthcare data — that's AES's job (Phase 4).

Viva Questions

Q: Why JWT?

JWT allows the server to authenticate subsequent API requests using a signed token without requiring the user to send their password repeatedly.

Q: Why BCrypt?

To securely hash passwords before storing them in the database.

Q: Authentication vs authorization?

Authentication verifies identity; authorization determines permissions.

Q: Why protect `/api/keys/**`?

Because key-management operations are sensitive and should only be accessible to authenticated and authorized users.

Phase 4 — AES-256 Encryption

The Problem

If sensitive information such as a patient name or medical report is stored as plain text, anyone who gains unauthorized access to the storage can read it directly. This is why encryption is required.

What Is Encryption?

Encryption converts readable **plaintext** into unreadable **ciphertext**. Only someone with the correct cryptographic key can decrypt it back.

"Medical Report" → Encryption → "8F3aK9x...."

Why AES?

AES = Advanced Encryption Standard, a widely used **symmetric** encryption algorithm — the same key is used to both encrypt and decrypt.

```
Plaintext
+
Secret Key
|
▼
Encryption
|
▼
Ciphertext
|
▼
Decryption (+ Same Secret Key)
|
▼
Plaintext
```

Why AES-256 Specifically?

AES supports AES-128, AES-192, and AES-256 — the number is the key size in bits. This project uses AES-256, meaning 256 bits = 32 bytes, which is why the code checks:

```
if (keyBytes.length != 32)
```

? Why not AES-128?

AES-128 is also considered secure for many applications — the answer isn't "AES-128 is insecure." Rather, AES-256 was chosen because this is a healthcare-oriented key-management project handling potentially highly sensitive data, and AES-256 provides a larger key space and is a strong, widely accepted choice for long-term protection.

? Why not AES-192?

AES-192 is also secure, but AES-256 is commonly preferred when you want a larger security margin and a straightforward 256-bit key-management design.

? Why not RSA?

AES	RSA
Symmetric (same key)	Asymmetric (public + private key)
Fast	Much slower for bulk data
Good for encrypting large amounts of data	Often used for key exchange / signatures

A common architecture uses AES to encrypt the actual data, and RSA/ECC to protect or exchange the AES key. For this phase, AES is appropriate because the focus is on data encryption.

Why AES-GCM Instead of Simple AES?

Options include AES/ECB, AES/CBC, and AES/GCM. This project uses:

```
AES/GCM/NoPadding
```

GCM provides **authenticated encryption**, which gives:

Property	What it means
1. Confidentiality	Attackers can't simply read the encrypted data
2. Integrity / authentication	If ciphertext is modified, the authentication check detects it during decryption



So the system isn't only hiding data — it's also **detecting tampering**.

What Is the IV?

```
byte[] iv = new byte[12];
secureRandom.nextBytes(iv);
```

The IV (Initialization Vector) ensures that encrypting the same plaintext with the same key does not normally produce the same ciphertext:

Encryption #1	Encryption #2
IV A → Ciphertext A	IV B → Ciphertext B

Result: Ciphertext A ≠ Ciphertext B. This is why the unit test specifically checks that encrypting the same value twice produces different ciphertext.

Why SecureRandom?

Ordinary random generators can be predictable. Cryptographic randomness needs to be difficult for an attacker to predict, which is exactly what `SecureRandom` provides.

Why Base64?

AES produces raw binary bytes, which aren't convenient to place directly into JSON, database fields, or HTTP responses. Base64 encodes those bytes into safe text.

⚠ Common viva trap: Base64 is **encoding**, not encryption. It provides no confidentiality on its own.

The Full Phase 4 Flow

Encryption	Decryption
1. Plaintext 2. AES-256 key 3. Generate random 12-byte IV 4. AES-GCM encryption 5. Ciphertext + authentication tag 6. Combine IV + encrypted data 7. Base64 encode 8. Stored / transmitted	1. Base64 input 2. Decode 3. Extract IV 4. AES-256 key 5. AES-GCM decryption 6. Authentication verification 7. Original plaintext

AES vs BCrypt vs JWT — Don't Mix Them Up

Technology	Purpose
BCrypt	Protect user passwords
JWT	Authenticate API requests
AES-256-GCM	Encrypt sensitive data
Base64	Encode binary data for transport/storage
SecureRandom	Generate unpredictable cryptographic values

Viva Questions

Q: Why AES-256? It provides strong symmetric encryption with a 256-bit key and is appropriate for protecting sensitive data.
Q: Why GCM? GCM provides authenticated encryption, giving both confidentiality and integrity protection.
Q: Why random IV? To prevent repeated encryption of the same plaintext with the same key from producing predictable/repeated ciphertext.
Q: Why Base64? To represent binary encrypted data safely in text-based formats such as JSON and database fields. Base64 itself is not encryption.
Q: Why not RSA for everything? RSA is asymmetric and comparatively expensive for bulk data encryption. AES is much more efficient for encrypting large amounts of data.

The One-Minute Explanation for Your Mentor

If your mentor says "Explain what you have implemented so far," here's a clean answer:

"We are developing a healthcare key-management backend using Spring Boot. In Phase 1 we established the backend foundation with MySQL and JPA. In Phase 2 we implemented the key-management foundation and key metadata such as key ID, algorithm, version, status, and lifecycle timestamps. In Phase 3 we added authentication and authorization using users, roles, BCrypt password hashing, and JWT-based security, and protected the key APIs. In Phase 4 we added AES-256-GCM encryption. We use AES because it is efficient for symmetric encryption of sensitive data, and GCM additionally provides integrity and authentication. We generate a random IV for every encryption operation and use Base64 to safely represent the resulting binary data."

What Comes Next: Phase 5

The project has now covered:

Phase 1	Backend foundation
---------	--------------------

Phase 2	Key management
---------	----------------

Phase 3	User / API security
---------	---------------------

Phase 4	Data encryption
---------	-----------------

One major question remains: **where do the actual cryptographic keys come from, how are they generated, how are they stored/managed, and how do we safely rotate them?** That's the subject of Phase 5.